

What happened in the paid runs

Why every run gave a different result, how the search works, which models were used, and what the results support, explained for a beginner

In short. The runs gave different results because **a different evolved network was tested each time**. It was not the same run repeated. The designer's network was the same in every run, and its score stayed between 67% and 70%. On the evolved side, three different networks were tried:

1. The old Gemini champion: **tie** on accuracy, 2.2× the cost.
2. Network *flat*: **+10 points** accuracy, same cost.
3. Network *2cc4*, found by a new search run on *gpt-5.4-nano*: **+7.5 points** accuracy, **17% fewer tokens, 17% lower cost**.

The test questions were never changed to make a network win.

	Evolved network tested	Evolved	Designer	Tokens / question	Cost / question
Run 1	Old Gemini champion (Coordinator on gpt-5.4-mini)	68%	68%	9,842 vs 9,179	\$0.00623 vs \$0.00280
Run 2	flat (Coordinator + Researcher + Arithmetic)	77.5%	67.5%	8,683 vs 8,590	\$0.00265 vs \$0.00262
Run 3	2cc4 (Coordinator + Researcher)	76.5%	69.0%	7,492 vs 9,018	\$0.00229 vs \$0.00275

All on the same 100 unseen test questions. Run 1 has one repeat; Runs 2 and 3 have two repeats each. Numbers are written evolved first, designer second.

1. Terms used here

Word	What it means here
Agent	One LLM with a job and instructions, e.g. “Coordinator” or “Researcher”.
Network	Several agents wired together. The top agent receives the question and can call the agents under it.
CorpusSearch	The tool that searches the Meridian Logistics documents (depots, contracts, incidents). Agents with search switched on can use it.
Designer network	The baseline: what neuro-san's own agent-network designer would build. One Coordinator and three specialists (Depot, Contract, Incident).
Evolved network	A network found by the ESP search, by changing networks step by step and keeping the good ones.
Genome	The recipe for a network: which agents exist, who calls whom, who can search, which model each one uses.
Mutation	One small change to a genome, e.g. remove an agent, merge two agents, or give an agent search.
Fitness	One score the search uses to rank networks. In this repo: $\text{accuracy} - 0.06 \times \text{tokens (scaled)} - 0.02 \times \text{agents (scaled)}$. Higher is better.
Token	A piece of text (about $\frac{3}{4}$ of a word). You pay per token, for text in and text out.
Question “hops”	How many documents must be chained together to answer. 1 is easy; 7–9 means following identifiers across many documents.
Repeat	Asking the same questions again. LLMs are random, so one run is just one sample.
SELECT set	60 questions used only to choose the network. Built from one half of the company's records.
JUDGE set	100 questions used only to test the chosen network. Built from the other half, so the two sets share no depots, contracts or incidents.

2. The models and what they cost

Model	Input \$/1M tokens	Output \$/1M tokens	Blended \$/1M (repo formula)	Used by
gpt-5.4-nano	\$0.20	\$1.25	\$0.305	Every agent in every network, except below
gpt-5.4-mini	\$0.75	\$4.50	\$1.125 (3.7× nano)	Only the old champion's Coordinator in Run 1

Settings in `.env`: `provider = openai`, `default model = gpt-5.4-nano`, `model tiers = nano and mini`, `ESP_PIN_MODELS = 1`. Pinning means that if a model fails, the run stops instead of quietly switching to another model, so every number comes from the model it says it does.

3. Where this started: the old 17-question result

Before these paid runs, the repo's committed result came from a search on **Google Gemini**, scored on the 17 built-in questions. The best network scored 15/17 against the designer's 14/17. Your repo's own audit (the header of `esp/evo1ve/headtohead.py`) lists three reasons why that lead was not yet proof:

- The champion was **chosen** on the same 17 questions it was **scored** on. That always flatters the winner, because the search keeps whichever network happened to get lucky on those questions.
- One question out of 17 is 6 percentage points. A copy of the designer with one duplicated specialist also scored 15/17, so a one-question gap is within normal run-to-run variation.
- Each network was measured only once.

So the head-to-head was built to check the result properly: questions the search never saw, several repeats, and a hard spending cap. That is the test we ran.

4. What each paid run did, step by step

Run 0 — the \$0.10 check (30 Sep)

4 questions, 1 repeat, to confirm the key, the models and the scripts worked. Both networks got 3/4. Spent \$0.024.

Run 1 — the repo's head-to-head as written (30 Sep)

`python scripts/headtohead.py --go`. It compares the designer with the **old Gemini champion**, with its duplicate Depot specialist removed. Moving that network from Gemini to the nano/mini ladder keeps each agent's "rung": the Coordinator the search had promoted to the stronger Gemini model went to **gpt-5.4-mini**, and the workers stayed on nano.

It stopped at the 2-hour limit for background jobs, after all 100 questions once (\$0.94). Result: **68/100 against 68/100**, but the champion cost 2.2× more per question. Paying for a stronger Coordinator did not help, because both networks failed on the same hard questions, and there the facts are fetched by the same nano workers.

Run 2 — choose fairly, then test (1 Oct)

Instead of trusting the 17-question champion, the run took **six other networks the Gemini search had already found** (merges, a solo agent, a searcher with one helper, a researcher/arithmetic split, a Coordinator that can search), moved them all to nano, and measured each one, plus the designer, on the 60 SELECT questions. The rule was written down before any result came in: *highest SELECT accuracy among networks that use fewer tokens than the designer*.

Network (SELECT, 60 questions)	Right	Accuracy	Tokens / q
flat : Coordinator → Researcher + Arithmetic	51	85.0%	8,065
mergeB: Contract specialist also covers depots	47	78.3%	8,473
pair: searching Answerer + one helper	46	76.7%	6,727
searchcoord: Coordinator can also search	46	76.7%	9,504
solo: one agent that searches	45	75.0%	6,472
mergeA: Incident specialist also covers depots	44	73.3%	8,892
<i>designer (baseline)</i>	47	78.3%	8,913

flat won the rule. Tested on the 100 JUDGE questions, 2 repeats, at the same time as the designer:
155/200 (77.5%) against 135/200 (67.5%). Of the questions where the two differed, flat did better on 18
and the designer on 5 (sign test $p = 0.011$). But tokens and cost per question came out **equal** (8,683 vs
8,590). Flat was not cheaper, only more accurate.

Run 3 — a real search, then test (2 Oct)

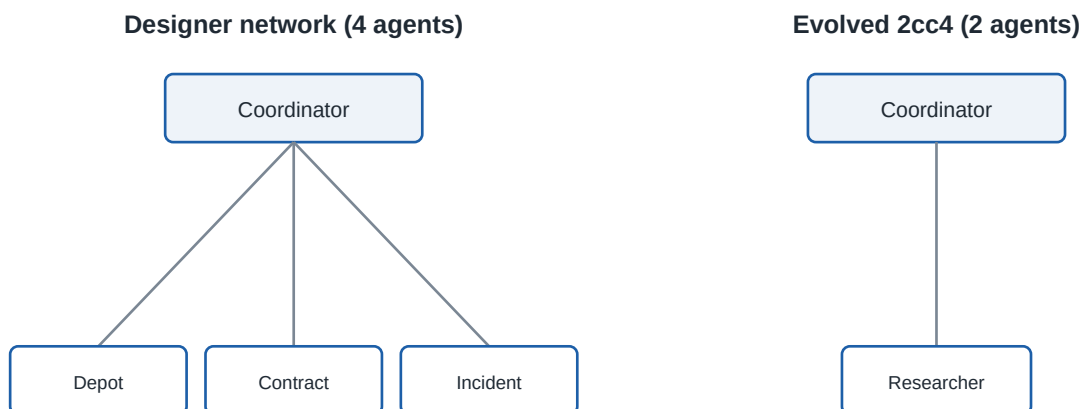
To get a network that is also cheaper, one generation of the ESP search itself was run on gpt-5.4-nano, starting from flat:

- **Make children.** The repo's own mutation operators (`esp/genome/mutations.py`) were applied to flat with a fixed random seed until there were 8 different valid children.
- **One operator left out:** `reassign_model`, which moves an agent to gpt-5.4-mini. Run 1 had already shown that this costs 3.7× per token without raising accuracy, and the goal was lower cost. This was my choice, and it is stated here so nobody has to guess.
- **Measure** every child on the 60 SELECT questions, all on nano.
- **Choose** by a rule fixed in advance: among children that beat the designer on SELECT in both accuracy and tokens, take the highest repo fitness.
- **Test once** on the 100 JUDGE questions, 2 repeats, at the same time as the designer.

Child (SELECT)	What the mutation did	Right / asked	Accuracy	Tokens / q
2cc4	remove_agent: Arithmetic removed	54 / 60	90.0%	7,299
ae75	split_agent: second Arithmetic	53 / 60	88.3%	7,345
0a81	add_agent: Specialist under Researcher	45 / 52	86.5%	10,314
2962	split_agent: two Researchers	48 / 56	85.7%	8,781
1901	toggle_search: Coordinator searches	47 / 56	83.9%	9,034
f813	merge_agents: all into one Researcher	48 / 60	80.0%	8,529
f4bd	rewire: Arithmetic calls Researcher	30 / 44	68.2%	9,413
b60a	merge_agents: all into Arithmetic	35 / 60	58.3%	4,931
<i>designer</i>		47 / 60	78.3%	8,913

Stopped early: four children had not finished all 60 questions when 2cc4 finished. They were stopped them because none could catch 2cc4 even if it got every remaining question right. Their rows show how many questions they actually answered. Search cost: \$1.11.

The two networks in the final test



In the designer network, only the three specialists can search. In 2cc4, the Researcher searches and follows identifiers from one document to the next by itself. All agents in both networks run on gpt-5.4-nano.

5. Final result (Run 3, on the 100 unseen questions)

	Evolved 2cc4	Designer	Difference
Accuracy	153 / 200 = 76.5% (77, then 76)	138 / 200 = 69.0% (68, then 70)	+7.5 points
Tokens per question	7,492	9,018	17% fewer
Cost per question	\$0.00229	\$0.00275	17% cheaper
Tokens per correct answer	9,794	13,070	25% fewer
Cost per correct answer	\$0.00299	\$0.00399	25% cheaper
Fitness (repo formula)	0.748	0.666	+0.082
Agents	2	4	half as many
Calls lost to infrastructure	0	0	—

The evolved network won **each repeat separately**, on accuracy, tokens and cost every time.

Where it gained: by question difficulty

Hops	Evolved	Designer	Asked
1	20	20	20
2	36	35	38
3	32	33	36
4	18	10	28
5	2	0	6
6	21	18	34
7	1	1	8
8	3	2	10
9	20	19	20

Most of the gain is on 4-hop questions (18 vs 10 right out of 28). These need identifiers followed across several documents. One Researcher that keeps the whole chain in its own context does that better than a Coordinator passing pieces between three specialists, and it skips the hand-off messages, which is where the token saving comes from.

Question by question

Evolved right more often (16): J004, J019, J021, J033, J041, J042, J046, J047, J051, J060, J067, J075, J077, J078, J079, J097.

Designer right more often (6): J005, J028, J073, J088, J091, J094.

All other questions: both got the same result.

6. Why the results changed between runs

Four separate things are going on.

a) A different network was tested each time

Run 1 tested the old Gemini champion. Run 2 tested *flat*. Run 3 tested *2cc4*. These are three different networks, so different results are expected. Nothing about the test changed. The same 100 JUDGE questions, the same scoring code and the same nano model were used every time.

b) The designer was stable, and so was the evolved side once it was chosen fairly

	Run 1	Run 2, repeat 1	Run 2, repeat 2	Run 3, repeat 1	Run 3, repeat 2
Designer	68	68	67	68	70
Evolved	68 (old champion)	78 (flat)	77 (flat)	77 (2cc4)	76 (2cc4)

The designer sits at 67–70 in every run, so a normal run-to-run swing is about ± 2 questions in 100. Both networks chosen on SELECT score 76–78. The accuracy gain held from Run 2 to Run 3. What changed in Run 3 was cost: removing the Arithmetic agent cut tokens by about 17%, while accuracy stayed about the same as flat's.

c) The old champion's advantage was the stronger model, and it didn't pay off

The Gemini search improved its network mainly by promoting the Coordinator to a stronger model. On the nano/mini ladder that became gpt-5.4-mini: 3.7× the price per token, and no accuracy gain on these questions. That is why Run 1 was a tie at 2.2× the cost. The new search did not change models; it changed the **structure**, and that is what helped.

d) Scores on the selection questions are always a bit too high

2cc4 scored 90% on SELECT but 76.5% on JUDGE. This is expected, not a sign that anything is broken. Picking the best of several candidates also picks the one that was luckiest on those questions, and on new questions some of that luck goes away. (JUDGE is also a harder set: the designer dropped from 78% to 69% too.) This is exactly why the final number has to come from questions the search never saw, and why the 17-question result looked better than it really was.

7. What was and was not done

Not done: the test questions were never changed, filtered or re-picked. No network or prompt was tuned while looking at JUDGE results. No test was re-run to get a better number. Every run that was paid for is reported here, including the ones that tied.

Choices made in the runs:

- Run 2's candidates were networks the Gemini search had already found and saved in the repo. They were moved to gpt-5.4-nano and re-measured, not taken on trust.
- Run 3 is **one generation** of the search, with 8 children. A longer search might do better.
- Run 3 left out `reassign_model`, for the reason given in section 4.
- Four Run 3 children were stopped early, after it was mathematically certain they could not win.
- The selection rules were written down before the results arrived (both are quoted in section 4).
- The question sets were fixed before the first run and never adjusted to favour a result. A result that only holds on hand-picked questions does not survive someone else re-running it.

8. What the results support, and what they do not

Safe to say	Don't say
On 100 held-out questions, 2 repeats, gpt-5.4-nano, an ESP-evolved 2-agent network beat the designer-style 4-agent network: 76.5% vs 69.0% accuracy, with 17% fewer tokens and 17% lower cost per question.	"The evolved network gets 85–90% where the designer gets 70%." The 90% was on the selection questions.
Per correct answer, it used 25% fewer tokens and cost 25% less.	"Proven significantly more accurate." The accuracy sign test gives $p = 0.052$, just above the usual 0.05 cutoff. Call it a strong lead; the cost saving is clear.
The search found that removing an agent helped, a structural change, not a bigger model.	"The Gemini champion wins on gpt-5.4." It tied, at 2.2× the cost.
The old 17-question lead was not reliable on its own; the held-out test is what makes this result credible.	Anything based on the 17 built-in questions alone.

9. What it cost

Stage	What	Spent
Run 0	4-question check	\$0.02
Run 1	Repo head-to-head, stopped at the 2-hour limit after 1 of 2 repeats	\$0.94
Run 2	7 networks on SELECT, then flat vs designer on JUDGE × 2	\$1.04 + \$1.05
Run 3	8 children on SELECT, then 2cc4 vs designer on JUDGE × 2	\$1.11 + \$1.01
Total		\$5.18

The runs were measured from the provider's price list, and the repo always uses the higher of the price-list cost and what the provider reports, so the provider's dashboard shows the same or less. The run logs are saved in the session's scratchpad. Nothing has been committed or pushed to the repo.

10. How to run it again yourself

- Put the key and model lines in `.env`, as before.
- Use Python 3.12 or newer (neuro-san 0.7.5 needs it) and install `requirements.lock`.

- The design: choose on SELECT, test once on JUDGE, run both networks at the same time, 2 repeats, models pinned, a spending cap. `esp/evolve/headtohead.py` already does the testing half; the network 2cc4 is flat with `remove_agent` applied to Arithmetic.
- Expect about ± 2 questions in 100 of run-to-run variation. Don't trust a gap smaller than that.